

DMCP Studio: An Interactive, Trace-Grounded Studio for Effect-Scored Evaluation of LLM Agents over Live MCP Servers

Jerzy Kamiński* Artem Kuznetsov Ilya Galyukshev[†] Sergey Chuprin

Kirill Redko Aidar Shumbalov Anna Kalyuzhnaya

ITMO University [†] Central University

Abstract

Benchmarks for LLM agents over Model Context Protocol (MCP) servers typically grade the final answer against a reference string or check a fixed “ground-truth” list of tools, both of which break once the underlying servers are live and stateful. We present **DMCP Studio**, an interactive system that makes an alternative visible and inspectable: *grade effects, not the answer*. The studio walks a user through the full pipeline in four live stages—collect MCP servers, explore a generated goal to record a successful trajectory, distill that trajectory into path-agnostic *effect checkpoints*, and score a candidate agent on whether it reproduces those effects. Before generation, a Benchmark Advisor turns the user’s evaluation intent into a statistically scoped plan, exposing the task budget, minimum detectable effect (MDE), assumptions, and claim boundaries the planned corpus can support. The signature interaction is an **Effect $\rightleftharpoons$ Answer toggle** that re-grades the same run two ways and *flips the verdict*, dramatizing failure modes that answer-matching gets wrong. A LIVE/REPLAY switch backs every stage with either real pipeline calls or frozen, deterministic fixtures, so the demo is reproducible on a booth laptop yet provably wired to the real system. We report a studio-vs-batch verdict-agreement study, a per-stage latency budget, and an informal usability pass. DMCP Studio turns a static evaluation methodology into a hands-on tool that agent builders and evaluation researchers can drive on their own MCP servers.

1 Introduction

Builders of LLM-agent systems increasingly design their products around MCP servers: file stores, databases, web services, developer platforms, and other external tools exposed behind a uniform calling interface. For such a system, the practical question is not whether an agent can issue a tool call in

isolation, but whether the configured agent–server stack accomplishes the user’s goal across many real, interacting services. Yet the benchmarks used to answer that question mostly score one of two proxies: the agent’s final natural-language answer, matched against a reference string, or the set of tools the agent was expected to call. **Both are fragile precisely where deployment matters.** A reference answer goes stale the moment it depends on live data—today’s price, this morning’s commit, the current contents of a table—and a fixed tool list cannot be recovered from the prompt alone, so it accumulates spurious “required” tools that penalize an agent for taking a different, equally valid route.

The companion methodology paper, DynamicMCPBench (Kamiński et al., 2026), argues that the organizing object of the benchmark should be neither the answer nor the tool list but the *execution trace*: explore a goal live, distill the successful trajectory into path-agnostic *effect checkpoints*, and grade a candidate purely on whether it reproduces those effects. That argument is easy to state and hard to feel. A reader of the paper sees tables; what they do not see is the moment a confident, fluent answer is marked **FAILED** because the agent never produced a required piece of evidence, or the moment a “wrong” answer is marked **SOLVED** because the live data simply moved.

DMCP Studio makes benchmark design and execution interactive. It wraps the existing DynamicMCPBench pipeline, helps the user decide what benchmark is statistically worth generating, and then walks that benchmark through four live stages—*collect* servers, *explore* a generated goal into a reference trace, *distill* that trace into a TaskSpec of effect checkpoints, and *score* a candidate against them (Figure 1). The pre-run *design* stage is not a passive explanation panel: Benchmark Advisor computes what claim the planned corpus can support, what task budget it requires, and what assumptions must remain visible before

*Corresponding author: jkamiński@niuitmo.ru

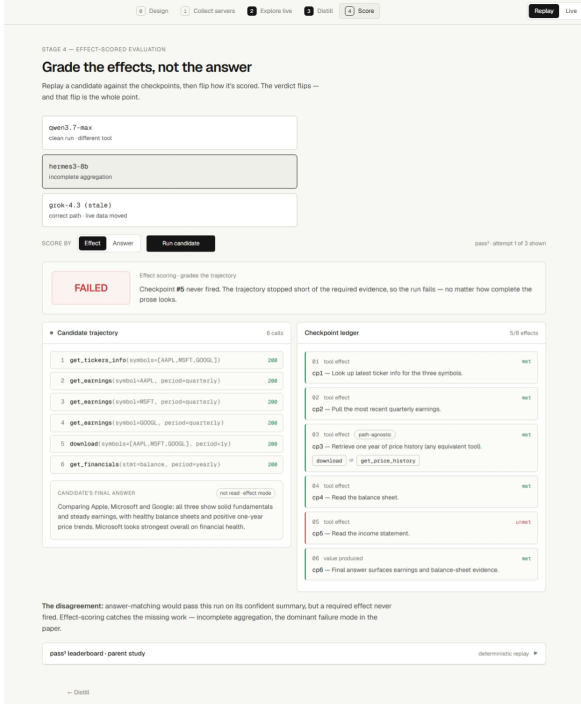


Figure 1: **DMCP Studio** (scoring stage, REPLAY mode). The four-stage stepper (collect → explore → distill → score) and the LIVE/REPLAY toggle sit in the header. Here the hermes3-8b candidate is marked **FAILED**: its trajectory (left) reproduces every effect *except* the income-statement checkpoint (#5, red in the ledger, right), even though its prose summary would pass answer-matching—the incomplete-aggregation case called out in the note below. The Effect ⇌ Answer toggle re-grades the *same* recorded run two ways without a second backend call, and the download/get_price_history equivalence set on checkpoint #3 is an editable chip the visitor can toggle to re-score live.

any corpus is generated (Stage 0, §3.2). The interactive centerpiece for scoring remains the **Effect ⇌ Answer toggle**, which grades one candidate run under both regimes and flips the verdict on screen. Three curated cases make the disagreement concrete: (1) a clean pass that reaches every effect through a *different but equivalent* tool; (2) a confident agent that passes answer-matching but *fails* effect-scoring because it skipped required work (incomplete aggregation); and (3) a correct agent that *fails* answer-matching but passes effect-scoring because the live data drifted since the reference was recorded.

Two design choices keep the demo both honest and useful as a system. First, a **LIVE/REPLAY** mode switch: **REPLAY** serves frozen fixtures deterministically—this is at once a booth-safety measure and on-thesis, since the methodology itself scores under deterministic replay—while **LIVE** drives the real pipeline against read-only servers as proof. Second, the studio never reimplements scoring: all real work flows through thin adapters over the published pipeline and the Advisor services, so what the visitor sees is a usable system rather than a mock-up.

Audience. DMCP Studio targets three groups: agent builders who need a deployment-specific diagnostic on their own MCP fleet, evaluation researchers comparing scoring methodologies, and practitioners who want to understand *why* answer-matching misleads on live, stateful tools before adopting an effect-based alternative.

Contributions. (i) An interactive benchmark-design and execution studio that exposes a trace-grounded, answer-agnostic evaluation pipeline end-to-end in the browser, with a verdict-flipping Effect/Answer toggle as its central scoring interaction (§3); (ii) Benchmark Advisor, a pre-generation statistical design assistant that maps user intent to a scoped benchmark plan, searches candidate budgets and effect targets, pre-computes the MDE implied by the unique-task budget, and carries only defensible plans toward corpus generation (§3.2); and (iii) a LIVE/REPLAY design plus a system evaluation: studio-vs-batch verdict agreement, per-stage latency, and an informal usability pass (§4).

2 Related Systems

DMCP Studio is, to our knowledge, the first interactive demonstration system built specifically

system	live servers	effect scored	forward gen.	inter- active
MCP batch benchmarks ^a	✓	—	—	—
Replay environments ^b	—	✓	—	—
Task generators ^c	✓	—	—	—
DMCP Studio	✓	✓	✓	✓

Table 1: **Where DMCP Studio sits.** *Effect-scored* = grades path-agnostic effects, not the answer or a tool list; *forward gen.* = tasks distilled from real trajectories rather than imposed graphs; *interactive* = a visual studio rather than a CLI/batch harness. ^aFan et al. (2025); Mo et al. (2025); Wang et al. (2025); ^bGuo et al. (2024); Lu et al. (2025); Yao et al. (2024); ^cLiu et al. (2025); Shen et al. (2024).

around *answer-agnostic, effect-based* grading of MCP agents over live servers. We situate it against three families of prior work; Table 1 summarizes the contrast.

MCP agent benchmarks. A large and fast-growing line evaluates agents directly on MCP servers, pushing breadth and realism (Fan et al., 2025; Mo et al., 2025; Lei et al., 2025; Wang et al., 2025) with execution-grounded scoring (Luo et al., 2025; Gao et al., 2025) or planted-distractor partial credit (Bandi et al., 2026), and others stress-test or diagnose failure (Wu et al., 2025; Yin et al., 2025; Guo et al., 2026b). These ship as *batch harnesses and fixed datasets*: a practitioner runs a script and reads a score. None exposes the construction and scoring of a task as an inspectable, interactive surface, and each ultimately grades a final answer, an outcome, or a tool selection rather than path-agnostic effects.

Reproducible tool-execution environments. Because live tools drift, a separate line makes execution reproducible—caching call responses behind a virtual server (Guo et al., 2024, 2025), retrieving similar cached responses on a miss (Cheng et al., 2025), or scoring milestones, forbidden actions, and dual-control state (Lu et al., 2025; Barres et al., 2025). Yao et al. (2024) compares final database state rather than the path. DMCP Studio adopts this deterministic-replay stance for its REPLAY mode, but turns it outward: replay is not only a stability mechanism, it is the thing the visitor watches and toggles.

Agent-evaluation tooling and generators. Toolkits such as Liu et al. (2025) generate and verify MCP tasks, and others build tool graphs and

back-instruct questions (Shen et al., 2024; Guo et al., 2026a; Shi et al., 2025). These are developer libraries run from a terminal. DMCP Studio differs in *modality* (an interactive, visual studio rather than a CLI) and in *thesis* (it dramatizes the gap between effect-scoring and answer-matching, a contrast a batch tool never surfaces).

In short, prior systems are batch harnesses, replay back-ends, or generator libraries; DMCP Studio is an interactive front-end that makes trace-grounded, effect-based evaluation *visible*—and, through its LIVE/REPLAY switch, both reproducible and demonstrably live. It is the companion demonstration to the DynamicMCPBench methodology (Kamiński et al., 2026); this paper describes the system and its evaluation, not the methodology, with which we keep overlap minimal.

3 System

3.1 Architecture

DMCP Studio is a static single-page front-end over a FastAPI back-end whose job is to wrap the published DynamicMCPBench pipeline and the Benchmark Advisor services through thin adapters. The end-to-end flow is: user intent → Advisor statistical plan → guarded corpus generation → collect/explore/distill/score. The adapter exposes one function per pipeline stage and returns the pipeline’s own pydantic models; the back-end never reimplements distillation or scoring, and the Advisor stays side-effect-free until the user confirms a guarded corpus-only handoff. The two slow stages—exploration and candidate scoring—stream call-by-call over Server-Sent Events (SSE), so the UI fills as the agent acts. This “wrap, don’t rewrite” boundary is what lets us claim the studio shows the real system: every verdict on screen is computed by the same code path the methodology paper reports, and every launchable design uses the same contract the generation pipeline consumes.

3.2 Stage 0: designing the benchmark

Benchmark Advisor is a pre-generation statistical design assistant (Figure 2): it decides what a planned benchmark can support before the user spends time generating a corpus. From a natural-language evaluation question plus optional structure (comparison mode, candidate models, task budget, attempts, requested detectable effect, server scope, and task mix) it returns a `StatisticalPlan`: a scoped claim, method choice, assumption ledger,

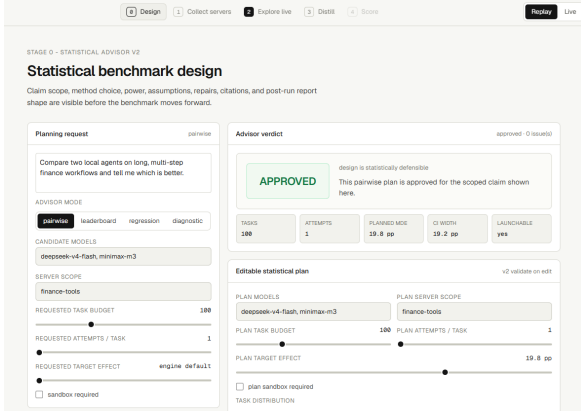


Figure 2: **Stage 0 — Design.** Benchmark Advisor turns an evaluation question into a statistically scoped benchmark plan before generation. The workbench shows the supported claim, MDE and power diagnostics, assumptions, repair actions, alternatives, and guide citations. Approved or warning-level plans can be carried forward only through an explicit guarded handoff to corpus/specs/traces generation.

power curve, issues and repairs, alternatives, and guide citations.

The design loop is guide-first and deterministic. A curated `STATISTICAL_GUIDE.md` maps intent signals to mode, metric, budget and repeat policy, criterion, and claim boundary; the engine then searches a finite grid of candidate budgets, attempt counts, and effect targets, validates each with deterministic rules, and labels it **approved**, **warning**, or **refused**. The recommendation is not a UI default but the cheapest sufficient candidate, with stronger and narrowed-claim alternatives kept visible. Its core diagnostic is the pre-run minimum detectable effect: for a binary effect-pass metric the Advisor uses $MDE \approx (z_{1-\alpha/2} + z_{1-\beta}) \sqrt{2p(1-p)/n_{\text{eff}}}$ (no-prior defaults $p = 0.5$, $\alpha = 0.05$, power 0.80). Crucially $n_{\text{eff}} \leq$ the number of *unique* tasks: repeated attempts support reliability analysis but do not multiply the independent sample size, so a 40-task smoke run and a 500-task selection run make visibly different claims before either is launched.

Launchability is separate from approval text. Refused or under-specified designs export nothing; approved or warning-level designs carry into Collect only through an explicit confirmation restricted to corpus/specs/traces generation. The Advisor never starts leaderboard evaluation or paid scoring—it gates and documents the benchmark the rest of the system will generate.

3.3 Four stages

1. Collect. The visitor points the studio at MCP servers—the three public servers from the worked example, or any server URL—and the collector returns each server’s tool surface tagged by dynamism (live-read vs. stateful-write). State-changing servers are marked and, per the sandbox rule (Ethical Considerations), never invoked unless flagged sandboxed. If an Advisor plan was carried forward, Collect uses its task budget, server scope, and task-distribution constraints as the generation contract.

2. Explore. A goal generator turns the selected tool surface into a realistic user request; an explorer agent then pursues it *live*, and every successful tool call streams into a reference trace. Generation is forward: we explore first and distill second, so no tool dependency graph is ever imposed. Tool names are visible to the explorer here but stripped from the candidate’s prompt later, so a candidate cannot copy the reference path.

3. Distill. Unlike a fixed tool-list grader, a checkpoint is not a required reference call: it is an observable effect with admissible realizations, argument constraints, and, when needed, value-level evidence. The distiller compiles the recorded trace into a path-agnostic *TaskSpec*: a set of effect checkpoints (a tool effect, demanding that *some* tool from an equivalence set ran with the right arguments, or a *value-produced* effect), optional forbidden-action minefields, and a partial order. Because every checkpoint comes from a tool that was actually used to reach the goal, a spurious “unnecessary tool” cannot appear. The studio renders each checkpoint as a card and—crucially—renders equivalence sets as **editable chips** the visitor can toggle.

4. Score. The visitor picks a candidate agent and watches its trajectory replay against the checkpoint ledger. The scoring endpoint returns *both* an effect verdict and an answer verdict on every call; the Effect/Answer toggle is therefore a pure front-end re-render over one payload, never a second back-end call. We define `answer_pass` as a demo-only answer-matching baseline: the candidate’s final message is compared with the recorded reference final answer after simple normalization; it is independent of the trace and is never used as the `DynamicMCPBench` verdict. Toggling an equivalence-set member re-scores Tier-1 effects live, so the visitor can watch a pass become a failure by disabling the

very tool the candidate happened to use.

3.4 The verdict flip

The studio is organized around making answer-matching and effect-scoring *disagree on screen*. Three curated cases (frozen as REPLAY fixtures, §4) each reproduce a different disagreement:

1. **Equivalent-tool pass.** The candidate reaches the price-history effect with `get_price_history` where the reference used `download`. Effect-scoring passes it via the equivalence set; a tool-list grader would flag a “wrong” tool.
2. **Answer-pass, effect-fail.** A fluent agent skips the income-statement step, then writes a confident summary. Answer-matching accepts the prose; effect-scoring marks the missing checkpoint `unmet`. This *incomplete-aggregation* failure is the dominant failure mode in the parent study.
3. **Answer-fail, effect-pass.** A correct agent reports live returns that no longer match the stored reference string. Answer-matching penalizes a genuinely correct run; effect-scoring passes it. This is exactly the fragility of grading the answer on live data.

3.5 LIVE vs. REPLAY

A mode switch in the header backs every stage with either real pipeline calls (LIVE) or frozen fixtures served deterministically (REPLAY). REPLAY is the public default: it makes the demo reproducible on a booth laptop and is on-thesis, because the methodology grades under deterministic replay. LIVE drives the real pipeline against read-only servers as proof, with per-stage timeouts and a graceful fall-back to the REPLAY fixture if a live server is unreachable, so a flaky network never bricks the demo. The headline verdict uses Tier-1 deterministic scoring (matching the paper); the optional Tier-2 effect-equivalence judge is shown as an upgrade, never on the critical path.

4 Evaluation

A demonstration paper must show that the system works, not re-establish the methodology (done in Kamiński et al., 2026). We therefore evaluate four properties of the *studio*: that its verdicts match the batch pipeline, that it is fast enough to drive live, that the Advisor enforces its planning assumptions

	comparisons	agreement
Overall verdict	118	100.0%
Per-checkpoint (Tier-1)	708	100.0%

Table 2: **E1: studio-vs-batch agreement.** Each of 118 deterministic (*spec*, *candidate*) pairs (22 pass / 96 fail) is scored through the studio backend and through the original batch pipeline (the `dmcp eval` CLI, a separate process). The studio’s JSON round-trip and result shaping reproduce the pipeline’s deterministic Tier-1 verdict exactly — overall and for every checkpoint — meeting the pre-registered 100% bar. Tier-2 (the non-deterministic LLM equivalence judge) is off the demo’s critical path.

before generation, and that visitors can reach the verdict flip unaided.

4.1 E1: Studio-vs-batch verdict agreement

The studio is only credible if it reproduces the pipeline’s verdicts. We score the same set of (*task*, *candidate*) pairs two ways—through the studio back-end and through the original batch pipeline—and measure agreement on the Tier-1 effect verdict, which is deterministic and so should agree exactly. Table 2 reports the result.

The pre-registered target is 100% Tier-1 agreement; any discrepancy indicates the adapter diverged from the pipeline and is a bug to fix, not a tolerance to accept. Across 118 deterministic pairs (22 pass, 96 fail) the studio reproduces the batch verdict exactly—both overall and for all 708 per-checkpoint comparisons—so the demo shows the pipeline’s own scorer, not a re-implementation. Tier-2 (the LLM effect-equivalence judge) is reported separately, since it is not deterministic and is off the demo’s critical path.

4.2 E2: Per-stage latency budget

Whether a stage runs LIVE or from a REPLAY fixture in the booth is decided by its wall-clock cost. We measure the per-stage *compute* of the REPLAY booth path through the real routes (inter-event pacing disabled, so we time the system rather than the cosmetic animation delay). Table 3 reports the budget.

Every stage is sub-millisecond and cold-start to first verdict is ≈ 1.3 ms—roughly four orders of magnitude under the 30 s target (§5)—so the only delay a visitor perceives is the deliberate, configurable inter-event pacing (~ 0.45 s/call). The decision rule follows: the booth runs REPLAY for every stage. LIVE exploration, which would be dominated by

stage	median	p95	booth
servers	0.66 ms	0.74 ms	replay
goal	0.47 ms	0.53 ms	replay
explore (SSE)	0.87 ms	0.96 ms	replay
distill	0.48 ms	0.52 ms	replay
score (SSE)	0.99 ms	1.06 ms	replay
evaluator / pair	0.021 ms	0.025 ms	—

Table 3: **E2: REPLAY booth-path latency** (request compute, SSE pacing disabled; $n=200$ /stage, evaluator over the 118-pair battery). Every stage is sub-millisecond and cold-start to first verdict is ≈ 1.3 ms — far under the 30 s target — so the booth runs REPLAY for every stage. The only time a visitor perceives is the deliberate, configurable inter-event pacing (~ 0.45 s/call). LIVE goal/explore (the proof path, not the booth’s graded path) are not timed here and would be dominated by LLM/network seconds.

LLM and network seconds, is offered as an explicit proof run rather than the graded path—consistent with scoring under deterministic replay throughout.

4.3 E3: Showcase fixtures

We curate and freeze the three verdict-flip fixtures of §3, plus the worked-example default (the AAPL/MSFT/GOOGL financial-comparison task): each is a candidate trajectory scored by the *real* Tier-1 evaluator (E1), so the demo deterministically reproduces each disagreement with the pipeline’s own verdicts rather than a re-implementation. Each fixture carries a one-line rationale and is served by the REPLAY back-end; LIVE re-derives the same TaskSpec against the read-only servers.

4.4 E4: Advisor planning behavior

Because Benchmark Advisor is a launch gate rather than a scorer, we evaluate it with an informal scenario-based planning suite. We constructed 20 evaluation intents covering pairwise comparisons, leaderboard construction, diagnostic smoke tests, underpowered claims, repeated-attempt misuse, and edited-plan revalidation. For each scenario we specified the expected mode, launchability status, required warning or repair, and one invariant over the planning diagnostics.

The Advisor selected the expected mode in 20/20 cases and the expected approved/warning/refused status in 20/20 cases. All underpowered or unsafe requests produced an explicit repair or narrower-claim alternative, and all refused plans exported no corpus handoff. In monotonicity checks, increasing the number of unique tasks always decreased the

planned MDE, while increasing attempts per task never increased the effective independent sample size. These checks do not prove that a future corpus is representative; rather, they verify that the Studio exposes and enforces the planning assumptions it claims to enforce before generation.

4.5 E5: Informal usability

We ran an informal pass with a handful of users unfamiliar with the system, observing whether they could reach the first verdict flip unaided and noting points of confusion; no formal user study and no quantitative metric are claimed. Participants reached the verdict flip without guidance, and their observations fed directly back into the interface: clearer labelling of the Effect/Answer toggle, an explicit equivalence-set hint, and a more prominent LIVE/REPLAY affordance.

5 Conclusion

DMCP Studio turns a static evaluation methodology—grade effects, not the answer—into something a visitor can drive. In four live stages it collects MCP servers, explores a goal into a reference trace, distills that trace into path-agnostic effect checkpoints, and scores a candidate against them; a single Effect $\Rightarrow$ Answer toggle then flips the verdict on the same run, making the failure of answer-matching on live, stateful tools impossible to miss. A LIVE/REPLAY switch keeps the demo deterministic and booth-safe while remaining provably wired to the published pipeline. We hope the studio lowers the barrier for agent builders and evaluation researchers to adopt effect-based grading on their own MCP fleets.

Limitations

Several limitations qualify the system. First, **live-server flakiness**: LIVE mode depends on third-party MCP servers that can be slow or unreachable, which is why REPLAY is the default and every LIVE stage falls back to a frozen fixture—so the most reliable path is not, strictly, live. Second, **scorer conservatism**: the studio inherits the methodology’s conservative effect scorer, which accepts a run only when the required effects are demonstrably present, so the verdicts shown are lower bounds on capability, not a definitive judgment of an agent. Third, **a single-task default showcase**: the booth centers on one worked example and three curated fixtures chosen to make

the verdict flip vivid, so it illustrates the disagreement rather than measuring its prevalence (the parent study does the latter). Fourth, **Tier-2 non-determinism**: the optional effect-equivalence judge is an LLM and is kept off the demo’s critical path; the headline verdict is Tier-1 only. Finally, **Benchmark Advisor is a planning assistant, not an oracle**: its MDE and power curves are conservative pre-run heuristics over declared assumptions, not post-run proof, and cannot guarantee that public-log priors transfer to a new MCP fleet or that missing outcomes are benign; final claims still depend on the observed outcomes, the missingness policy, and the claim boundary declared before generation.

Availability and License

A live, no-login demo runs in deterministic REPLAY at <https://huggingface.co/spaces/jrzkaminski/dmcp-studio>. DMCP Studio also ships as a self-contained, installable package: from a clean checkout, `docker build -f dmcp-studio/Dockerfile -t dmcp-studio .` followed by `docker run -p 8000:8000 dmcp-studio` brings the studio up at `http://localhost:8000` in under ten minutes, with no API keys and no network (the public default is the deterministic REPLAY path).¹ A screen-cast (<2.5 min) walks through the four stages and the verdict flip.² The studio and the underlying pipeline are released under the Apache-2.0 license; the released datasets are distributed under their respective source-server terms.

Ethical Considerations

The studio executes real tools during LIVE exploration. To prevent side effects, the adapter enforces a default-deny rule: no tool on a state-changing (stateful_write) server is invoked unless that server is explicitly flagged sandboxed, and exploration and scoring otherwise operate only against read-only servers, so no real external state is modified. REPLAY mode touches no network at all. The system handles no personal data; the public servers in the default showcase expose only public market, scholarly, and encyclopedic data. The work conforms to the ACM Code of Ethics.

¹Source and install instructions accompany the hosted Space (its Files tab).

²Screen-cast: <https://www.youtube.com/watch?v=66AI1SURCA8>.

Use of Large Language Models

LLMs are the objects of study (the agents under evaluation), components of the pipeline the studio wraps (the goal generator, the explorer agent, and the optional Tier-2 effect-equivalence judge), and were additionally used to assist with copy-editing this manuscript. All system design, experiments, and analysis are the authors’ own.

References

- Chaithanya Bandi, Ben Hertzberg, Geobio Boo, Tejas Polakam, Jeff Da, Sami Hassaan, Manasi Sharma, Andrew Park, Ernesto Hernandez, Dan Rambado, and 1 others. 2026. Mcp-atlas: A large-scale benchmark for tool-use competency with real mcp servers. *arXiv preprint arXiv:2602.00933*.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.
- Xiang Cheng, Yulan Hu, Xiangwen Zhang, Lu Xu, Zheng Pan, Xin Li, and Yong Liu. 2025. Travelbench: A real-world benchmark for multi-turn and tool-augmented travel planning. *arXiv preprint arXiv:2512.22673*.
- Shiqing Fan, Xichen Ding, Liang Zhang, and Linjian Mo. 2025. Mcptoolbench++: A large scale ai agent model context protocol mcp tool use benchmark. *arXiv preprint arXiv:2508.07575*.
- Xuanqi Gao, Siyi Xie, Juan Zhai, Shiqing Ma, and Chao Shen. 2025. Mcp-radar: A multi-dimensional benchmark for evaluating tool use capabilities in large language models. *arXiv preprint arXiv:2505.16700*.
- Xiaojie Guo, Yang Zhang, Bing Zhang, Ryo Kawahara, Mikio Takeuchi, and Yada Zhu. 2026a. Uni-toolbench: A benchmark for tool-augmented llms in cross-domain, universal task automation. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 4726–4736.
- Zhicheng Guo, Sijie Cheng, Yuchen Niu, Hao Wang, Sicheng Zhou, Wenbing Huang, and Yang Liu. 2025. Stabletoolbench-mirrorapi: Modeling tool environments as mirrors of 7,000+ real-world apis. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 5247–5270.
- Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. 2024. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 11143–11156.

- Zikang Guo, Benfeng Xu, Chiwei Zhu, Wentao Hong, Xiaorui Wang, and Zhendong Mao. 2026b. Mcp-agentbench: Evaluating real-world language agent performance with mcp-mediated tools. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 30888–30896.
- Jerzy Kamiński, Ilya Galyukshev, Artem Kuznetsov, Sergey Chuprin, Kirill Redko, Aidar Shumbalov, and Anna Kalyuzhnaya. 2026. DynamicMCPBench: A trace-grounded, effect-scored benchmark for LLM agents over live MCP servers. Manuscript under review.
- Fei Lei, Yibo Yang, Wenxiu Sun, and Dahua Lin. 2025. Mcpverse: An expansive, real-world benchmark for agentic tool use. *arXiv preprint arXiv:2508.16260*.
- Zhiwei Liu, Jieli Qiu, Shiyu Wang, Jianguo Zhang, Zuxin Liu, Roshan Ram, Haolin Chen, Weiran Yao, Shelby Heinecke, Silvio Savarese, and 1 others. 2025. Mcpeval: Automatic mcp-based deep evaluation for ai agent models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 373–402.
- Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, and 1 others. 2025. Tool-sandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1160–1183.
- Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao, Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo, Silvio Savarese, Caiming Xiong, and Junnan Li. 2025. MCP-Universe: Benchmarking large language models with real-world model context protocol servers. *arXiv preprint arXiv:2508.14704*.
- Guozhao Mo, Wenliang Zhong, Jiawei Chen, Qianhao Yuan, Xuanang Chen, Yaojie Lu, Hongyu Lin, Ben He, Xianpei Han, and Le Sun. 2025. Livemcpbench: Can agents navigate an ocean of mcp tools? *arXiv preprint arXiv:2508.01780*.
- Yongliang Shen, Kaitao Song, Xu Tan, Wenqi Zhang, Kan Ren, Siyu Yuan, Weiming Lu, Dongsheng Li, and Yueting Zhuang. 2024. Taskbench: Benchmarking large language models for task automation. *Advances in Neural Information Processing Systems*, 37:4540–4574.
- Dingfeng Shi, Jingyi Cao, Qianben Chen, Weichen Sun, Weizhen Li, Hongxuan Lu, Fangchen Dong, Tianrui Qin, King Zhu, Minghao Liu, and 1 others. 2025. Taskcraft: Automated generation of agentic tasks. *arXiv preprint arXiv:2506.10055*.
- Zhenting Wang, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, Alireza Rezazadeh, Ankit Shah, Yujia Bao, and 1 others. 2025. Mcp-bench: Benchmarking tool-using llm agents with complex real-world tasks via mcp servers. *arXiv preprint arXiv:2508.20453*.
- Zijian Wu, Xiangyan Liu, Xinyuan Zhang, Lingjun Chen, Fanqing Meng, Lingxiao Du, Yiran Zhao, Fanshi Zhang, Yaoqi Ye, Jiawei Wang, and 1 others. 2025. Mcpmark: A benchmark for stress-testing realistic and comprehensive mcp use. *arXiv preprint arXiv:2509.24002*.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*.
- Ming Yin, Dinghan Shen, Silei Xu, Jianbing Han, Sixun Dong, Mian Zhang, Yebowen Hu, Shujian Liu, Simin Ma, Song Wang, and 1 others. 2025. Livemcp-101: Stress testing and diagnosing mcp-enabled agents on challenging queries. *arXiv preprint arXiv:2508.15760*.

A Worked Example: the Verdict Flip

The default showcase compares the financial health of Apple (AAPL), Microsoft (MSFT), and Google (GOOGL): latest ticker info, most recent quarterly earnings, and one year of price history. The explorer records a successful trajectory over the yfinance server, which the distiller compiles into six checkpoints—five tool effects (one carrying a {download, get_price_history} equivalence set) and one value-produced effect over the final message. Table 2 and the three fixtures of §4 are drawn from this example.

B API Contract

The back-end exposes six pipeline routes; `mode ∈ {live, replay}` defaults to replay. The Stage-0 Benchmark Advisor adds a parallel POST `/api/advisor/design/validate` pair— and a `/v2/` statistical-workbench family—that plan and re-validate a design without touching the pipeline routes below.

- GET `/api/servers` — collected servers with dynamism tags.
- POST `/api/goal` — generate a goal from a tool surface.
- GET `/api/explore` — SSE stream of explorer calls.
- POST `/api/distill` — compile a trace into a TaskSpec.
- GET `/api/score` — SSE stream of candidate calls and per-checkpoint verdicts; the done event returns *both* `effect_pass` and `answer_pass`, so the Effect/Answer toggle is a pure front-end re-render.

- GET /api/leaderboard — parent-study results for the peek panel (loaded from the real study, never invented).